데이터베이스 최적화 (2)

MySQL

박연종

2026.03.25.

System Software Lab.

목차

0 1 데이터 세트 소개

0 2 문제 상황

0 3 해결 전략

데이터 세트

NYC Taxi and Limousine Commission에서 제공하는 2025년 Yellow Taxi Trip Records 사용 (1월 ~ 11월 내역)

약 78,039,389개 레코드가 존재하며, 용량은 약 8.6GB

예시 데이터

데이터 제공자 (제공자 ID)	미터기 시작 시각	미터기 종료 시각	승객 수	여행 거리 (mile)	미터기 시작 구역 (구역 ID)	미터기 종료 구역 (구역 ID)	요금 코드 (코드 ID)	저장 및 전송 플래그	결제 유형 (유형 ID)	금액 ($)	기타 요금 ($)	교통국 세금 ($)	개선 운임 ($)	팁 금액 ($)	톨게이트 금액 ($)	총 금액 ($)	미터기 시작 연도	미터기 시작 달
2	2088년 1월 24일 오전 12:25:39	2088년 1월 24일 오전 7:28:25	1	4.05	24	162	1	N	2	14.5	0	0.5	0.3	0	0	15.3	2088	1
2	2088년 1월 24일 오전 12:15:42	2088년 1월 24일 오전 12:19:46	1	0.63	41	166	1	N	2	4.5.	0	0.5	0.3	0	0	5.3	2088	1
2	2084년 11월 4일 오후 12:32:24	2084년 11월 4일 오후 12:47:41	1	1.34	238	236	1	N	2	10	0	0.5	0.3	0	0	10.8	2084	11
2	2084년 11월 4일 오후 12:25:53	2084년 11월 4일 오후 12:29:00	1	0.32	238	238	1	N	2	4	0	0.5	0.3	0	0	4.8	2084	11

비즈니스 로직

특정 일로부터 과거 3개월 간 정상 운행 건의 총 매출을 구하기

```
SELECT SUM(total_amount) AS ‘total_sales_3m’
  FROM yellow_trip
 WHERE payment_type IN (1,2)
   AND fare_amount > 0
   AND total_amount > 0
   AND trip_distance > 0
   AND tpep_pickup_datetime >= @adt - INTERVAL 3 MONTH;
```

EXPLAIN 명령

```
EXPLAIN [ANALYZE] <QUERY>
```

id	쿼리 번호	단일 쿼리면 1만 존재하나 서브쿼리 등이 포함되면 1,2,3,…
select_type	SELECT 쿼리 유형	SIMPLE, PRIMARY, UNION, SUBQUERY, DERIVED, …
table	SELECT 대상 테이블	FROM의 테이블 또는 <union qid#1, qid#2>
partitions	파티션	NULL 또는 파티션 이름
type	데이터 접근 방법	…, range(인덱스 특정 범위), index(인덱스 전체), ALL(풀 테이블 스캔)
possible_keys	인덱스 후보군	사용 후보로 선택된 인덱스 집합 (사용하지 않을 수 있음)
key	실제 선택된 인덱스	NULL이면 인덱스를 사용하지 않은 것임
key_len	인덱스 길이	데이터 접근 방법
ref	인덱스 사용 시 비교법	const(상수), db.<table>.<column>(컬럼 값), func(함수 결과), NULL
rows	읽어야 하는 행 수	
filtered	읽은 행 중 조건에서 산 행의 비율	Rows가 1000이고, filtered가 50.00이면 500행이 조건문에서 살아남을 것이라 예상
Extra	추가 실행 정보	Using where, Using index, …

쿼리 실행 전 쿼리 평가

특정 일로부터 과거 3개월 간 정상 운행 건의 총 매출을 구하기

```
SELECT SUM(total_amount) AS ‘total_sales_3m’
  FROM yellow_trip
 WHERE payment_type IN (1,2)
   AND fare_amount > 0
   AND total_amount > 0
   AND trip_distance > 0
   AND tpep_pickup_datetime >= @adt - INTERVAL 3 MONTH;
```

id	1
select_type	SIMPLE
table	yellow_trip
partitions	<null>
type	ALL
possible_keys	idx_tpep_pickup_datetime
key	<null>
key_len	<null>
ref	<null>
rows	75689795
filtered	0.31
Extra	Using where

쿼리 실행

	total_sales_3m
1	316142375.56

1 row retrieved starting from 1 in 2 m 18 s 971 ms (execution: 2 m 18 s 611 ms, fetching: 360 ms)

쿼리 분석

```
-> Aggregate: sum(yellow_trip.total_amount)  (cost=8.08e+6 rows=1) (actual time=78058..78058 rows=1 loops=1)

    -> Filter: ((yellow_trip.payment_type in (1,2)) and (yellow_trip.fare_amount > 0.00) and
(yellow_trip.total_amount > 0.00) and (yellow_trip.trip_distance > 0.00) and
(yellow_trip.tpep_pickup_datetime >= <cache>(((@anchor_dt) - interval 3 month))) and
(yellow_trip.tpep_pickup_datetime < <cache>((@anchor_dt))))
       (cost=8.06e+6 rows=233477) (actual time=0.452..77818 rows=10.7e+6 loops=1)

        -> Table scan on yellow_trip  (cost=8.06e+6 rows=75.7e+6) (actual time=0.43..67059 rows=77.6e+6 loops=1)
```

왜 테이블 풀 스캔을 했는가?

```
SELECT SUM(total_amount) AS 'total_sales_3m'
  FROM yellow_trip
 WHERE payment_type IN (1,2)
   AND fare_amount > 0
   AND total_amount > 0
   AND trip_distance > 0
   AND tpep_pickup_datetime >= @adt - INTERVAL 3 MONTH;
```

인덱스가 존재하지 않음

인덱스가 존재함

MySQL의 옵티마이저가 실행 계획을 세울 때 인덱스를 사용하는 것보다 테이블 전체를 읽는 것이 더 이득이라 판단

테이블 레코드 수가 적어 인덱스를 사용하는 것보다 빠른 경우

WHERE 절이나 ON 절에서 인덱스를 사용할 수 있는 적절한 조건이 없는 경우

인덱스 범위 스캔을 사용할 수 있다 하더라도 조건에 일치하는 레코드 건 수가 너무 많은 경우

왜 테이블 풀 스캔을 했는가?

만약 인덱스를 사용한다면

인덱스를 읽으며 후보 엔트리를 찾은 후,

후보 엔트리에 존재하는 행들의 ID로 실제 행을 읽고,

WHERE 조건을 검사

다음 후보 엔트리에 대해 이를 반복

현재 인덱스는 미터기 시작 시각과 종료 시각 컬럼에만 설정돼 있음

payment_type, fare_amount, trip_distance, total_amount는 인덱스의 컬럼이 아니므로 실제 행을 읽어 재검사 필요

옵티마이저는 인덱스를 사용해 접근 시에도 찾아야 하는 행의 수가 좁혀지지 않기 때문에 테이블 풀 스캔을 택

인덱스 추가로 개선할 수 있다

결제 방식 및 시간 컬럼에 대한 인덱스

```
CREATE INDEX idx_yellow_trip_payment_pickup
ON yellow_trip (payment_type, tpep_pickup_datetime);
```

```
completed in 3 m 16 s 356 ms
```

```
1 row retrieved starting from 1 in 1 m 7 s 128 ms (execution: 1 m 6 s 637 ms, fetching: 491 ms)
```

```
-> Aggregate: sum(yellow_trip.total_amount)  (cost=8.15e+6 rows=1) (actual time=66487..66487 rows=1 loops=1)

    -> Filter: ((yellow_trip.payment_type in (1,2)) and (yellow_trip.fare_amount > 0.00) and
(yellow_trip.total_amount > 0.00) and (yellow_trip.trip_distance > 0.00) and
(yellow_trip.tpep_pickup_datetime >= <cache>(((@anchor_dt) - interval 3 month))) and
(yellow_trip.tpep_pickup_datetime < <cache>((@anchor_dt))))  (cost=8.12e+6 rows=233477) (actual
time=7.81..66247 rows=10.7e+6 loops=1)

        -> Table scan on yellow_trip  (cost=8.12e+6 rows=75.7e+6) (actual time=7.79..55830 rows=77.6e+6 loops=1)
```

인덱스 추가로 개선할 수 있다

쿼리 전용 인덱스

```
CREATE INDEX idx_yellow_trip_report
ON yellow_trip (
     payment_type,
     tpep_pickup_datetime,
     fare_amount,
     trip_distance,
     total_amount
);
```

```
completed in 4 m 28 s 456 ms
```

```
1 row retrieved starting from 1 in 6 s 532 ms (execution: 6 s 191 ms, fetching: 341 ms)
```

```
-> Aggregate: sum(yellow_trip.total_amount)  (cost=4.5e+6 rows=1) (actual time=6155..6155 rows=1 loops=1)
    -> Filter: ((yellow_trip.payment_type in (1,2)) and (yellow_trip.fare_amount > 0.00) and
(yellow_trip.total_amount > 0.00) and (yellow_trip.trip_distance > 0.00) and
(yellow_trip.tpep_pickup_datetime >= <cache>(((@anchor_dt) - interval 3 month))) and
(yellow_trip.tpep_pickup_datetime < <cache>((@anchor_dt))))  (cost=4.42e+6 rows=801109) (actual
time=0.314..5912 rows=10.7e+6 loops=1)
        -> Covering index range scan on yellow_trip using idx_yellow_trip_report over (payment_type = 1 AND
'2025-03-13 21:00:00.000000' <= tpep_pickup_datetime <= '2025-06-13 21:00:00.000000' AND 0.00 < fare_amount) OR
(payment_type = 2 AND '2025-03-13 21:00:00.000000' <= tpep_pickup_datetime <= '2025-06-13 21:00:00.000000' AND
0.00 < fare_amount)  (cost=4.42e+6 rows=21.6e+6) (actual time=0.304..3745 rows=10.8e+6 loops=1)
```

무분별한 인덱스 추가는 권장하지 않는다

테이블에 행이 추가되거나, 인덱스로 설정된 컬럼의 값이 변경되거나, 행이 삭제되는 경우

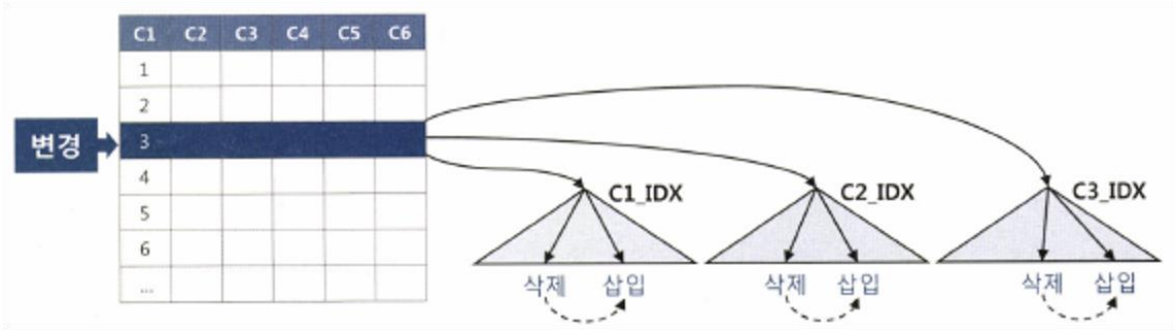
인덱스에 대해 재배치 작업이 발생

테이블에 인덱스가 많은 경우

각 인덱스들이 재배치 작업을 수행하고,
데이터베이스는 각 인덱스의 무결성 유지를 위해 잠금을 걸기 때문에 쓰기 작업 속도가 느려짐

인덱스도 저장 공간을 차지하므로 무분별한 인덱스 추가는 디스크 공간을 낭비할 수 있음

옵티마이저가 인덱스 후보군 선택할지 판단이 느려질 수 있음



인덱스 추가 없이도 성능을 어느 정도 개선할 수 있을까?

시간의 순서에 따라 주요 키(기본 키, Primary Key)가 증가하는 경우

특정 일로부터 과거 3개월에 대한 시작 PK를 찾고,

그 후 PK를 증가시키면서 실제 행을 읽으면 더 빠르게 집계할 수 있음

다만 현재 데이터베이스는 PK가 임의로 설정되어 시간의 순서에 따라 증가하지 않으므로

재현할 수 없었음

궁극적인 해결

기법분류	기법	내용
테이블병합	1:1 관계 테이블병합	1:1 관계를 통합하여 성능향상
	1:M 관계 테이블병합	1:M 관계 통합하여 성능향상
	슈퍼/서브타입 테이블병합	슈퍼/서브 관계를 통합하여 성능향상
테이블분할	수직분할	칼럼단위의 테이블을 디스크 I/O를 분산처리 하기 위해 테이블을 1:1로 분리하여 성능향상(트랜잭션의 처리되는 유형을 파악이 선행되어야 함)
	수평분할	로우 단위로 집중 발생되는 트랜잭션을 분석하여 디스크 I/O및 데이터접근의 효율성을 높여 성능을 향상하기 위해 로우단위로 테이블을 쪼갬(관계가 없음)
테이블추가	중복테이블 추가	다른 업무이거나 서버가 다른 경우 동일한 테이블구조를 중복하여 원격조인을 제거하여 성능을 향상
	통계테이블 추가	SUM, AVG 등을 미리 수행하여 계산해 둠으로써 조회 시 성능을 향상
	이력테이블 추가	이력테이블 중에서 마스터 테이블에 존재하는 레코드를 중복하여 이력테이블에 존재하는 방법은 반정규화의 유형
	부분테이블 추가	하나의 테이블의 전체 칼럼 중 자주 이용하는데 자주 이용하는 집중화된 칼럼들이 있을 때 디스크 I/O를 줄이기 위해 해당 칼럼들을 모아놓은 별도의 반정규화된 테이블을 생성

통계 테이블을 추가하고, 주기적으로 다시 집계해 통계 테이블에 삽입하거나 특정 시점을 저장 후 그 이후 데이터를 증분 반영하는 방법을 택이는 성능을 위해 의도적으로 데이터의 중복을 최소화하는 정규화를 위배하는 반정규화.

들어 주셔서 감사합니다

박연종

E-mail	park@duck.com
LinkedIn	yeonjong-park
Instagram	yeonjong.park
GitHub	patulus

2026.03.25.